



Reporte Técnico de Actividades Práctico- Experimentales Nro. 002

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Wilson Joel Guevara Moncada Yuleisy Cecibel Jaramillo Jaramillo Josselin Estefania Nero Fernandez Sdier Emanuel Roblez Roblez
Asignatura	Matemáticas Discretas
Ciclo	1er Ciclo
Unidad	Unidad 1: Lógica Matemática
Resultado de aprendizaje de la unidad	Desarrolla habilidades de razonamiento lógico para solucionar problemas ligados a la ingeniería, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001 fase 2
Título de la Práctica	Proposiciones, Conectores Lógicos, Inferencia, Deducción y Tablas de Verdad
Nombre del Docente	Mario Enrique Cueva Hurtado
Fecha	06 de abril al 15 de mayo de 2026
Horario	Lunes, martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación
Tiempo planificado en el Sílabo	18 horas (3 horas por sesión APE)

2. Objetivo(s) de la Práctica

- Identificar y clasificar proposiciones lógicas simples y compuestas, distinguiendo entre proposiciones y enunciados no proposicionales.
- Comprender y aplicar los conectores lógicos fundamentales (negación, conjunción, disyunción, condicional, bicondicional) en la formulación de expresiones lógicas.
- Construir tablas de verdad para proposiciones compuestas, determinando su valor de verdad en todos los casos posibles.
- Aplicar las leyes de inferencia y deducción lógica para resolver problemas de razonamiento lógico en contextos computacionales y de ingeniería.
- Desarrollar habilidades de trabajo colaborativo, pensamiento crítico y resolución de problemas, integrando principios de equidad, inclusión y responsabilidad ética en el aprendizaje.



3. Materiales, Reactivos, Equipos y Herramientas

- Pizarra y marcadores
- Proyector multimedia
- Hojas de trabajo impresas con ejercicios de lógica proposicional y tablas de verdad
- Guía de ejercicios de inferencia y deducción lógica
- Calculadora científica
- Material bibliográfico de apoyo (textos digitalizados de referencia)
- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL para entrega de evidencias
- Entorno virtual de aprendizaje (EVA-UNL)
- Herramientas colaborativas: Google Drive, Wiki
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

4. Procedimiento / Metodología Ejecutada

FASE 2: Construcción de Tablas de Verdad y Operaciones Combinadas (Semana 2: 13-17 abril 2026)

- Crear tablas de verdad de 2, 3, 4 variables identificando si es una tautología/contradicción/contingencia.
- Resolución ejercicios a través de la simplificación de expresiones lógicas aplicando las leyes fundamentales (ley de identidad, complemento, idempotencia, conmutativa, asociativa, distributiva, de Morgan, absorción).
- Analizar cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación.
- Entrega del primer portafolio de ejercicios individuales (propuestas y tablas de verdad) a través de la plataforma EVA.
- Discusión sobre la ética en el uso de la lógica para la toma de decisiones algorítmicas y su impacto en la sociedad.

5. Resultados

- Los estudiantes construyen tablas de verdad para expresiones con 2, 3 y 4 variables proposicionales, identificando los casos en que la expresión resulta verdadera, falsa o es una tautología/contradicción/contingencia.**

Ejercicios de 2 variables:

1) $(p \wedge \neg q) \vee (\neg p \wedge (p \vee q))$

	(	p	^	¬q	)	∨	(	¬p	^	(	p	∨	q	)	)
	V	F	F		F		F	F		V	V	V			
	V	V	V		V		F	F		V	V	F			
	F	F	F		V		V	V		F	V	V			
	F	F	V		F		V	F		F	F	F			

El ejercicio es una contingencia



2) $(p \vee q) \wedge (\neg p) \wedge (\neg q)$

$(p \vee q)$	$\wedge$	$(\neg p)$	$\wedge$	$(\neg q)$
V	V	F	F	V
V	F	F	V	F
F	V	V	F	V
F	F	V	V	F

Es una contradicción

Ejercicios de 3 variables:

3) $(p \vee q) \wedge (\neg p \vee r) \wedge (\neg q \vee r) \wedge \neg r$

$(p \vee q)$	$\wedge$	$(\neg p \vee q)$	$\wedge$	$(\neg q \vee r)$	$\wedge$	$\neg r$
V	V	F	V	V	F	V
V	F	F	V	F	F	V
V	V	F	F	V	F	V
V	F	F	V	F	F	V
F	V	V	V	V	F	V
F	V	V	F	F	F	V
F	F	V	V	V	F	V
F	F	V	F	F	F	V

Es una contradicción

4) $[(p \rightarrow q) \wedge (q \rightarrow r)] \rightarrow (p \rightarrow r)$

$[(p \rightarrow q) \wedge (q \rightarrow r)]$	$\rightarrow$	$(p \rightarrow r)$
V	V	V
F	V	F
V	V	V
F	V	F
V	V	V
F	V	F
V	V	V
F	V	F
V	V	V

Es una Tautología

Ejercicios de 4 variables:

$[(p \rightarrow q) \wedge (q \rightarrow r) \wedge (r \rightarrow s)] \rightarrow (p \rightarrow s)$

$[(p \rightarrow q) \wedge (q \rightarrow r) \wedge (r \rightarrow s)]$	$\rightarrow$	$(p \rightarrow s)$
V	V	V
F	V	F
V	V	V
F	V	F



unl

Universidad
Nacional
de Loja

		V	F	F		F		F	V	V		F		V	V	V		V		V	V	V	
		V	F	F		F		F	V	V		F		V	F	F		V		V	F	F	
		V	F	F		F		F	V	F		F		F	V	V		V		V	V	V	
		V	F	F		F		F	V	F		F		F	V	F		V		V	F	F	
		F	V	V		V		V	V	V		V		V	V	V		V		F	V	V	
		F	V	V		V		V	V	V		F		V	F	F		V		F	V	F	
		F	V	V		F		V	F	F		F		F	V	V		V		F	V	V	
		F	V	V		F		V	F	F		F		F	V	F		V		F	V	F	
		F	V	F		V		F	V	V		V		V	V	V		V		F	V	V	
		F	V	F		V		F	V	V		F		V	F	F		V		F	V	F	
		F	V	F		V		F	V	F		V		F	V	V		V		F	V	V	
		F	V	F		V		F	V	F		V		F	V	F		V		F	V	F	
		F	V	F		V		F	V	F		V		F	V	V		V		F	V	V	
		F	V	F		V		F	V	F		V		F	V	F		V		F	V	F	

Es una Tautología

2. **Práctica grupal:** cada equipo resuelve ejercicios de simplificación de expresiones lógicas aplicando las leyes fundamentales (ley de identidad, complemento, idempotencia, conmutativa, asociativa, distributiva, de Morgan, absorción).

PRIMER EJERCICIO:

$$[(p \vee \neg q) \wedge q] \vee \neg p$$

$$[(p \wedge q) \vee (\neg q \wedge q)] \vee \neg p \text{ Ley Distributiva}$$

$$[(p \wedge q) \vee F] \vee \neg p \text{ Ley de Complemento}$$

$$(p \wedge q) \vee \neg p \text{ Ley de Identidad}$$

$$(\neg p \vee p) \wedge (\neg p \vee q) \text{ Ley Distributiva}$$

$$V \wedge (\neg p \vee q) \text{ Ley de Complemento}$$

$$\neg p \vee q \text{ Ley de Identidad}$$

Es una contingencia

Tabla se verdad

[(	p	$\vee$	$\neg q$	)	$\wedge$	q	] $\vee$	$\neg p$
	V	V	F		V	V	V	F
	V	V	V		F	F	F	F
	F	F	F		F	V	V	V
	F	V	V		F	F	V	V

SEGUNDO EJERCICIO:

$$[(p \vee q \vee r) \vee (\sim p \wedge \sim q \wedge \sim r)] \wedge [(q \wedge r) \vee (q \wedge \sim r) \vee (\sim q \wedge r)] \leftrightarrow (q \vee r)$$

$$[\sim(\sim p \vee \sim q \vee \sim r) \vee (\sim p \wedge \sim q \wedge \sim r)] \wedge [(q \wedge r) \vee (q \wedge \sim r) \vee (\sim q \wedge r)] \leftrightarrow (q \vee r) \quad \mathbf{L. Morgan}$$

$$1 \wedge [(q \wedge r) \vee (q \wedge \sim r) \vee (\sim q \wedge r)] \leftrightarrow (q \vee r) \quad \text{L. Complemento}$$

$$1 \wedge [((q \wedge r) \vee (q \wedge \sim r)) \vee (\sim q \wedge r)] \leftrightarrow (q \vee r) \quad \mathbf{L. Associativa}$$

$$1 \wedge [(q \wedge (r \vee \sim r)) \vee (\sim q \wedge r)] \leftrightarrow (q \vee r) \quad \mathbf{L. Distributiva}$$

$$1 \wedge [(q \wedge 1) \vee (\sim q \wedge r)] \leftrightarrow (q \vee r) \quad \mathbf{L. Complemento}$$

$$1 \wedge [(q \vee (\sim q \wedge r)) \leftrightarrow (q \vee r)] \text{ **L. Propiedad**}$$

$$1 \wedge [(q \vee \sim q) \wedge (q \vee r)] \leftrightarrow (q \vee r) \quad \mathbf{L. Distributiva}$$

$$1 \wedge [1 \wedge (q \vee r)] \Leftrightarrow (q \vee r) \quad \text{L. Complemento}$$

$$1 \wedge [q \vee r] \Leftrightarrow (q \vee r)$$

$$(q \vee r) \Leftrightarrow (q \vee r)$$

V

1

Es una Tautología

Tabla de verdad

	$((p \vee q \vee r) \vee (\neg p \vee \neg q \vee \neg r)) \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$																				
	V	V	V	V	V	F	F	F	F	F	V	V	V	V	V	F	F	V	F	V	V
	V	V	V	F	V	F	F	F	F	V	V	F	F	V	V	V	V	F	F	F	V
	V	V	F	V	V	F	F	F	F	F	V	F	F	V	F	F	F	V	V	V	V
	V	V	F	V	F	V	F	F	F	F	F	F	F	F	F	V	F	V	F	F	V
	F	V	V	V	V	V	F	F	F	F	V	V	V	V	V	F	F	V	F	V	V
	F	V	V	V	F	V	V	F	F	F	V	V	F	F	V	V	V	V	F	F	F
	F	F	F	V	V	V	V	F	F	F	V	F	F	V	F	F	F	V	V	V	V
	F	F	F	F	F	V	V	V	V	V	F	F	F	F	F	V	F	F	F	V	F

TERCER EJERCICIO:

$$[(p \wedge \neg q) \vee p] \wedge \neg(\neg p \vee q)$$

$[p \wedge \neg(\neg p \vee q)] \wedge \neg(\neg p \vee q)$ Ley de absorción

$[p \wedge (p \vee \neg q)] \wedge (p \wedge \neg q)$ Ley de Morgan

$$p \wedge (p \wedge \neg q)$$

$(p \wedge p) \wedge \neg q$ Ley asociativa

$p \wedge \neg q$ Ley de idempotencia

Tabla de verdad

[(	p	^	¬q	)	v	p	]	^	¬	(	¬p	)	q	)
	V	F	F		V	V		F	F		F		V	V
	V	V	V		V	V		V	V		F		F	F
	F	F	F		F	F		F	F		V		V	V
	F	F	V		F	F		F	F		V		V	F

Es una contingencia



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

CUARTO EJERCICIO

$$[\neg((p \wedge q) \vee (p \wedge \neg q)) \vee (r \wedge \neg r)] \vee [(p \vee q) \wedge (p \vee \neg q) \wedge (p \vee r)]$$

$$[\neg(p \wedge (q \vee \neg q)) \vee (r \wedge \neg r)] \vee [(p \vee (q \wedge \neg q)) \wedge (p \vee r)]$$

$$[\neg(p \wedge 1) \vee 0] \vee [(p \vee 0) \wedge (p \vee r)]$$

$$[\neg p \vee 0] \vee [p \wedge (p \vee r)]$$

$$\neg p \vee [(p \vee p) \wedge (p \vee r)]$$

$$\neg p \vee [p \wedge (p \vee r)]$$

$$\neg p \vee [(p \wedge p) \vee (p \wedge r)]$$

$$\neg p \vee [p \vee (p \wedge r)]$$

$$\neg p \vee p$$

$$\vee$$

$$1$$

Es una Tautología

Tabla de verdad:

$\neg$	$(p \wedge q)$	$\vee$	$(p \wedge \neg q)$	$\vee$	$(r \wedge \neg r)$	$\vee$	$[(p \vee q) \wedge (p \vee \neg q) \wedge (p \vee r)]$
F	V	V	V	V	F	F	V
F	V	V	V	V	F	F	V
F	V	F	F	V	V	V	V
F	V	F	F	V	V	V	V
V	F	F	F	F	F	F	F
V	F	F	F	F	F	F	F
V	F	F	F	V	F	F	V
V	F	F	F	V	V	V	V

QUINTO EJERCICIO:

$$(\neg p \vee q) \wedge (p \vee q) \rightarrow q$$

$$q \vee (\neg p \wedge p) \rightarrow q \text{ ley distributiva}$$

$$q \vee F \rightarrow q \text{ ley de complemento}$$

$$q \rightarrow q \text{ ley de identidad}$$

$$\neg q \vee q \text{ ley de complemento}$$

$$\vee$$

Tabla de verdad

p	q	$\neg p$	$(\neg p \vee q)$	$(p \vee q)$	$(\neg p \vee q) \wedge (p \vee q)$	$(\neg p \vee q) \wedge (p \vee q) \rightarrow q$
V	V	F	V	V	V	V
V	F	F	F	V	F	V
F	V	V	V	V	V	V
F	F	V	V	F	F	V

3. Los equipos analizan problemas de lógica computacional: cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación (if-else, while, operadores lógicos en Python/C/Java).

I. Concepto de lógica booleana.

Según Días [1] "En la lógica booleana, desarrollada por George Boole en el siglo XIX, es una forma de álgebra donde una variable proposicional, como variable discreta, solo puede tener dos valores: Verdadera (V) o Falsa (F)".

Si relacionamos este concepto con programación podemos decir que sin la lógica booleana el software de cualquier computadora no podría analizar los algoritmos ya que es el puente entre el lenguaje humano y el de la computadora (la cual solo entiende 1 que es True y 0 que es False).

II. Operadores lógicos

Los operadores lógicos permiten evaluar diferentes condiciones actuando como conectores entre dos expresiones ya unidas por operadores racionales.

AND (Y - &&): Dos o más señales de entrada y aplicarle la función matemática de multiplicación para obtener un 1 lógico de salida, todas las señales de entrada deben de ser un 1 lógico.

OR (|| / O): Toma dos o más señales de entrada y aplicarle la función matemática de suma, para obtener un 1 lógico de salida, cualquiera las señales de entrada deben de ser un 1 lógico [2].

NOT (! / NO): Su funcionalidad se basa en cambiar su valor lógico [2], decir si la señal de entrada es un uno lógico, al aplicarle la función NOT se obtendría un cero lógico y así a la inversa.

Nota: Estos son los principales operadores lógicos sin embargo estos se pueden unir para formar otros operadores lógicos como **NAND** (fusión entre NOT y AND) el **NOR** (fusión entre NOT y OR) y **XOR** (fusión entre AND y OR, se necesita que los datos de entrada sean iguales para que nos del valor 1).

III. Operadores relacionales

Los operadores relacionales son aquellos que conectan o comparan datos, mediante estos se crea los valores de verdad.

- Igual a (==)
- Distinto de (!= o <>)
- Mayor que (>)
- Menor que (<)
- Mayor o igual que (>=)
- Menor o igual que (<=)

IV. Estructuras de control de flujo

Las estructuras de control de flujo son aquellos que ponen en marcha a la lógica booleana en el código, entre ellos tenemos:

Estructura selección (if-else):

Evalúa una condición y selecciona entre dos posibles caminos de ejecución.
Lenguaje de programación C.

En este programa se puede observar dos operadores lógicos “AND” y “OR”

```
C if_else_and.c
1  #include <stdio.h>
2  int main(){
3      //Variables
4      int edad;
5      char genero;
6
7      //Datos de entrada
8      printf("Ingrese su edad\n");
9      scanf("%i",&edad);
10
11     printf("Ingrese su genero M (Mujer) o H (Hombre)\n");
12     scanf(" %c",&genero);
13
14     //Proceso
15     if(edad >= 18 && (genero == 'M' || genero == 'm')){
16         printf("\nRESULTADO: Cumple con los requisitos de entrda. BIENVENIDA\n");
17     } else {
18         printf("\nRESULTADO: No cumple con los requisitos para el ingreso de esta reunión");
19     }
20
21     return 0;
22 }
```

OR: Este operador en el programa permite múltiples opciones válidas (genero == 'M' || genero == 'm'). Si representamos esto en la tabla de verdad podemos analizar las siguientes posibilidades:

M	m	M m	Procesamiento de la computadora
V	V	V	Como la proposición verdadera la computadora lo ejecuta o analiza como 1
V	F	V	Como la proposición verdadera la computadora lo ejecuta o analiza como 1 (solo se necesita que una de las opciones se cumpla).
F	V	V	Como la proposición verdadera la computadora lo ejecuta o analiza como 1 (solo se necesita que una de las opciones se cumpla).
F	F	F	La proposición es falsa por lo que la computadora lo analiza como 0 (solo si el usuario no ingresa ninguna de las variables la computadora la analiza como no cumplida).

AND: Necesita que las dos variables sean verdaderas par que toda la proposición sea verdadera (edad >= 18 && (genero == 'M' || genero == 'm')).

edad	genero	edad && genero	Interpretación
V	V	V	Solo cuando las dos variables cumplen con las condiciones establecidas la preposición se vuelve verdadera y la computadora la analiza como 1.
V	F	F	Una de las variables no cumple con las condiciones establecidas por lo que la proposición es falsa por lo que la computadora lo analiza como 0.
F	V	F	Una de las variables no cumple con las condiciones establecidas por lo que la proposición es falsa por lo que la computadora lo analiza como 0.
F	F	F	Una de las variables no cumple con las condiciones establecidas por lo que la proposición es falsa por lo que la computadora lo analiza como 0.

Resumen: En el código "if" significa en español "Si", lo que hace esto es que si las preposiciones que están dentro del paréntesis es verdadero (lo cual está definido por los conectores lógicos, en este caso AND y OR) se lleva acabo, una acción determinada pero si la primera condición no se cumple pasa a la acción determinada por "else" imprimiendo dicha opción.

Estructura de repetición (while):

Ejecuta una condición mientras sea verdadera y solo se detiene cuando esta sea falsa.

En el lenguaje de programación C se representa de la siguiente manera:

```
C while.c
1  #include <stdio.h>
2  int main(){
3      int clave=0;
4
5      while(!(clave==7)){
6          printf("Escribe la clave secreta para salir: ");
7          scanf("%i",&clave);
8      }
9
10     printf("!Clave correcta; Saliendo...");
11
12
13     return 0;
14 }
```

En este programa se puede ver el conector lógico NOT



unl

Universidad
Nacional
de Loja

NOT: Este operador lógico permite cambiar el valor de la verdad a una proposición. En el programa permite que while (ciclo) siga funcionando ya que dice que si la clave es diferente a 7 tendrá el valor de verdad de FALSO por lo que el ciclo terminaría, ¡pero si usamos ! la proposición sería verdadera por lo que el ciclo continúa hasta que la clave sea 7 (verdadera) y se convierta en F concluyendo con el ciclo.

Tabla de verdad:

clave	clave==7	!(clave==7)	Análisis de la Lógica Booleana
7	V	F (0)	La computadora detecta que la condición de parada se cumplió, el ! invierte el éxito a "falso" y el ciclo se detiene.
Cualquier otro número	F	V (1)	La computadora ve que la igualdad no se cumple, el ! lo cambia a "verdadero" y el ciclo continúa repitiéndose.

Entrega del primer portafolio de ejercicios individuales (propuestas y tablas de verdad) a través de la plataforma EVA. Discusión sobre la ética en el uso de la lógica para la toma de decisiones algorítmicas y su impacto en la sociedad.

Wilson Joel Guevara Moncada

https://drive.google.com/drive/folders/1tdXQw-N7CONka_MFNTkq5pjDLfb4Lv1u?usp=drive_link

Yuleisy Cecibel Jaramillo Jaramillo

<https://drive.google.com/drive/folders/1XJbLCJ7BFcFIdl5f5y0AV-CI9TN73vUf?usp=sharing>

Josselin Estefania Nero Fernandez

<https://drive.google.com/drive/folders/1ikMzsla8NV3VoyBBp3UvMxRvElKm5j8f?usp=sharing>

Sdier Emanuel Roblez Roblez

https://drive.google.com/drive/folders/1qZRz5aD9BuJ34NFqf4iOy5UdYO1_XdIJ?usp=sharing

6. Preguntas de Control

1. Explique la diferencia entre una proposición simple y una proposición compuesta. ¿Por qué es importante distinguir entre ambas en el contexto de la programación y diseño de algoritmos? Argumente con ejemplos concretos en lenguajes de programación.

La proposición simple es una sola oración sin conectores lógicos a diferencia de las proposiciones compuestas que está conformada por dos o más oraciones (proposiciones simples) que están conectadas por conectores lógicos (no, y, o, si...entonces, si...solo si).

Ambas son importantes en la programación ya que son base del pensamiento computacional y la toma de decisiones en los algoritmos, pero al no identificarlas correctamente puede ocasionar errores de ejecución, deficiencia en los programas y dificultades en la lectura y procesamiento, esto se debe, a que las proposiciones simples definen condiciones básicas y la compuestas construye lógica compleja.

Proposición simple:

Es una proposición simple ya que es una comparación

```
C proposicionesSimplesCompuestas.c
1  #include <stdio.h>
2  int main(){
3      //Variables
4      int edad;
5
6      //Datos de entrada
7      printf("ingrese su edad\n");
8      scanf("%i",&edad);
9
10     //Una proposición simple es una sola comparación
11     if (edad < 18){
12         printf("Usted es menor de edad\n");
13     }
14
15     return 0;
16 }
```

Proposiciones compuestas:

```
C proposicionesCompuestas.c
1  #include <stdio.h>
2  int main(){
3      // Datos de entrada
4      int edad;
5      char tutorLegal;
6
7      printf("Ingrese su edad\n");
8      scanf("%i",&edad);
9
10     printf("Ingresa si esta con un adulto o ingrese N si no lo esta\n");
11     scanf(" %c",&tutorLegal);
12
13     if(edad>=18 || tutorLegal=='S'){
14         printf("\nPuede acceder al lugar");
15     }else{
16         printf("Su acceso es denegado");
17     }
18     return 0;
19 }
```

Este programa es una proposición compuesta porque has dos comparaciones o variables ("edad>=18" y "tutorLegal=='S'") y estas a su vez se relacionan mediante "o" (||) que es un conector lógico).

7. Conclusiones

- Las tablas de verdad permiten obtener las respuestas (valor de verdad) de las expresiones lógicas de una manera muy ordenada y fácil de comprender, de igual forma al haber expresiones lógicas complejas y extensas permite estructurarlas, desglosarlas y verificarlas.
- Al aplicar las leyes deducción lógica e inferencia se puede simplificar o resolver las expresiones lógicas, en otras palabras, las vuelve más sencillas y fáciles de manejar.

8. Reflexión crítica

A través de esta actividad se ha desarrollado y comprendido mejor lo que son las tablas de verdad permitiendo la resolución y comprobación del valor de verdad de las operaciones lógicas.

También se ha mejorado la simplificación de operaciones lógicas mediante la aplicación de las leyes de proposiciones o leyes de deducción lógica a través de la práctica.



unl

Universidad
Nacional
de Loja

9. Bibliografía/Referencias

- [1] Díaz, L. (2022, noviembre 15). Una Aplicación Práctica De La Lógica Proposicional Y El Álgebra De Boole A Circuitos Serie-Paralelo. Eleutheria. <https://eleutheria.ufm.edu/una-aplicacion-practica-de-la-logica-proposicional-y-el-algebra-de-boole-a-circuitos-serie-paralelo>
- [2] M. T. J. Ramírez, «Compuertas lógicas,» Ingeniería eléctrica , 2020.
- [3] P. J. S. Yero, E. C. Alfonso y Y. R. Rodríguez, «Tratamiento metodológico a la integración de los operadores lógicos y relacionales con expresiones lógicas en la estructura selectiva doble,» *Opuntia Braba* , pp. 310-319, 11 Septiembre 2020
- [4] IBM, "Lógica booleana," IBM Documentation, 2024. [En línea]. Disponible en: <https://www.ibm.com/docs/es/i/7.5?topic=definitions-boolean-logic>. [Accedido: 23-abr-2026].
- [5] Microsoft, "Operadores lógicos y bit a bit en Visual Basic," Microsoft Learn, 2023. [En línea]. Disponible en: <https://learn.microsoft.com/es-es/dotnet/visual-basic/programming-guide/language-features/operators-and-expressions/logical-and-bitwise-operators>. [Accedido: 23-abr-2026].
- [6] W3Schools, "Python Booleans," W3Schools Online Web Tutorials, 2024. [En línea]. Disponible en: https://www.w3schools.com/python/python_booleans.asp. [Accedido: 23-abr-2026].



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

10. Anexos

Josselin Estefania Nero Fernandez

UNIVERSIDAD NACIONAL DE LOJA
MATEMÁTICAS DISCRETAS
APE 2

Nombre: Josselin Estefania Nero Fernandez
 Fecha: 23-04-2026
 Carrera-Cdo: Computación - 1º ciclo "A"
 Docente: Ing. Mario Cueva.

ESERCICIOS 2 VARIABLES.

1. $(p \wedge \neg q) \vee (\neg p \wedge (p \vee q))$

El ejercicio es una contingencia.

p	$\neg q$	$\vee$	$\neg p$	$\wedge$	p	$\vee$	q
V	F	F	F	F	V	V	V
V	V	V	F	F	V	V	F
F	F	F	V	V	F	V	V
F	F	V	V	F	F	F	F

2. $(p \vee q) \wedge (\neg p) \wedge (\neg q)$

Es una contradicción.

p	$\vee$	q	$\wedge$	$\neg p$	$\wedge$	$\neg q$
V	V	V	F	F	F	F
V	V	F	F	F	F	V
F	V	V	F	V	F	V
F	F	F	F	V	V	F

ESERCICIOS 3 VARIABLES

3. $(p \vee q) \wedge (\neg p \vee r) \wedge (\neg q \vee r) \wedge \neg r$

Es una contradicción.

p	$\vee$	q	$\wedge$	$\neg p$	$\vee$	r	$\wedge$	$\neg q$	$\vee$	r	$\wedge$	$\neg r$
V	V	V	V	F	V	V	F	V	V	V	F	F
V	V	V	V	F	V	V	F	V	F	F	F	F
V	V	F	F	F	V	F	F	V	V	V	F	F
V	V	F	F	F	V	F	F	V	F	F	F	F
F	V	V	V	V	F	V	V	F	V	V	F	F
F	V	V	V	V	F	V	V	F	V	F	F	F
F	F	F	F	V	F	V	F	V	V	V	F	F
F	F	F	F	V	F	V	F	V	F	F	F	F

4. $[p \rightarrow q] \wedge (q \rightarrow r) \rightarrow (p \rightarrow r)$

$p \rightarrow q$	$\wedge$	$q \rightarrow r$	$\rightarrow$	$p \rightarrow r$
V	V	V	V	V
V	V	F	V	F
V	F	F	V	V
V	F	V	V	F
F	V	V	V	V
F	V	F	V	F
F	F	V	V	V
F	F	F	V	V

Es una
Tautología.

EXERCICIOS 4 VARIABLES.

$[p \rightarrow q] \wedge (q \rightarrow r) \wedge (r \rightarrow s) \rightarrow (p \rightarrow s)$

$p \rightarrow q$	$\wedge$	$q \rightarrow r$	$\wedge$	$r \rightarrow s$	$\rightarrow$	$p \rightarrow s$
V	V	V	V	V	V	V
V	V	V	F	V	V	F
V	V	F	F	V	V	V
V	V	F	F	F	V	F
V	F	V	V	V	V	V
V	F	V	F	V	V	F
V	F	F	V	V	V	V
V	F	F	F	V	V	V
F	V	V	V	V	V	V
F	V	V	F	V	V	F
F	V	F	V	V	V	V
F	V	F	F	V	V	V
F	F	V	V	V	V	V
F	F	V	F	V	V	V
F	F	F	V	V	V	V
F	F	F	F	V	V	V

Es una
Tautología.

2. Ejercicios de simplificación de expresiones lógicas aplicando leyes fundamentales

$[p \vee \neg q] \wedge q \vee \neg p$

$[p \wedge q] \vee (\neg q \wedge q) \wedge \neg p$ Ley Distributiva

$[p \wedge q] \vee F \wedge \neg p$ Ley de Complemento

$(p \wedge q) \vee \neg p$ Ley de Identidad

$(\neg p \vee p) \wedge (\neg p \vee q)$ Ley Distributiva

$V \wedge (\neg p \vee q)$ Ley de Complemento

$\neg p \vee q$ Ley de Identidad

Tabla de verdad.

$p \wedge \neg q$	$\vee p$	$\wedge$	$\sim$	$\neg p$	q
V F F	V V	F	F	F V V	
V V V	V V	V	V	F F F	
F F F	F F	F	F	V V V	
F F V	F F	F	F	V V F	

Es una contingencia

Cuarto Ejercicio.

$$[\sim((p \wedge q) \vee (p \wedge \neg q)) \vee (r \wedge \sim r)] \vee [(p \vee q) \wedge (p \vee \neg q) \wedge (p \vee r)]$$

$$[\sim(p \wedge (q \vee \neg q)) \vee (r \wedge \sim r)] \vee [(p \vee (q \wedge \neg q)) \wedge (p \wedge r)]$$

$$[\sim(p \wedge 1) \vee 0] \vee [(p \vee 0) \wedge (p \vee r)]$$

$$[\sim p \vee 0] \vee [p \wedge (p \vee r)]$$

$$\sim p \vee [p \vee p \wedge (p \vee r)]$$

$$\sim p \vee [p \vee (p \vee r)]$$

$$\sim p \vee [(p \vee p) \vee (p \vee r)]$$

$$\sim p \vee [p \vee (p \vee r)]$$

$$\sim p \vee p$$

$$\vee$$

$$1$$

Tabla de verdad.

$\sim p \wedge q$	$\vee p \wedge \sim q$	$\vee \vee \wedge \sim$	$\vee$	$p \vee q$	$\wedge p \vee \sim q$	$\wedge p \vee \vee$
F V V V V F F V V F F	V	V V V V V F F	V	V V V V V V F V V	V V V V	
F V V V V V F F V V F F	V	V V V V V F F	V	V V V V V V F V V	V V V F	
F V F F V V V V V F F F	V	V V F V V V V V V V	V	V V F V V V V V V V	V V V F	
F V F F V V V V V F F V	V	V V F V V V V V V V	V	V V F V V V V V V F	V V V F	
V F F V F F F F F V F F	V	F V V F F F F F F F	V	F V V F F F F F F V	F F F F	
V F F V F F F F F V F F	V	F V V F F F F F F F	V	F V V F F F F F F F	F F F F	
V F F F F F F F F V F F	V	F F F F F F F F F F	V	F F F F F F F F F F	F F F F	
V F F F F F F F F V F F	V	F F F F F F F F F F	V	F F F F F F F F F F	F F F F	

Quinto Ejercicio.

$$(\sim p \vee q) \wedge (p \wedge q) \rightarrow q$$

$$q \vee (\sim p \wedge p) \rightarrow q \text{ Ley Distributiva}$$

$$q \vee F \rightarrow q \text{ Ley Complemento}$$

$$q \rightarrow q \text{ Ley de Identidad}$$

$$\sim q \vee q \text{ Ley de Complemento}$$



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Tabla de Verdad.

p	q	$\sim p$	$(\sim p \vee q)$	$(p \vee q)$	$(\sim p \vee q) \wedge (p \vee q)$	$(\sim p \vee q) \wedge (p \vee q) \rightarrow q$
V	V	F	V	V	V	V
V	F	F	F	V	F	V
F	V	V	V	V	V	V
F	F	V	V	F	F	V

ESTIL

3. Los equipos analizan problemas de lógica computacional

Según Dias [4] "En la lógica booleana, desarrollada por George Boole en el siglo XIX, es una forma de álgebra donde una variable proposicional, como variable discreta, solo puede tener dos valores: Verdadero (V) o Falso (F)".

Si relacionamos este concepto con programación podemos decir que sin la lógica booleana el software de cualquier computadora no podría analizar los algoritmos. Ya que es el puente entre el lenguaje humano y el de la computadora (la cual solo entiende 1 que es True y 0 que es False).

Es decir, que al momento de programar si definimos una variable y le damos una condición para que pueda tomar un valor, en programar si definimos solo hay 2 salidas posibles que la condición se cumpla y la variable tome el valor definido por lo cual, la computadora interpretará el resultado del algoritmo como 1 (True), o que la condición no se cumpla y la variable no tome el valor y la computadora lo interprete como 0 Falso.

AND (&): Su función lógica sería tomar dos o mas señales de entrada y aplicarle la función matemática de multiplicación. Su funcionalidad se basa en que, para obtener un 1 lógico de salida, todos los señales de entrada deben de ser un lógico en el momento que una de todos los señales de entrada sea un 0 lógico, su resultado será un 0 [2].

Entrada		Salida
A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

OR (||): Toma dos o mas señales de entrada y aplicarle la función matemática de suma, para obtener un 1 lógico de salida, cualquiera los señales de entrada deben de ser un lógico [2].

Entrada		Salida
A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

NOT (! / NO): Su funcionalidad se basa en cambio el valor lógico [2], decir si la señal de entrada es un uno lógico, al aplicarle la función NOT se obtendrá un cero lógico y así a la inversa.

Entrada	Salida
A	$\bar{A}$
0	1
1	0

Nota. Estos son los principales operadores lógicos sin embargo estos se pueden unir para formar otros operadores lógicos como NAND (fusión entre NOT y AND) el NOR (fusión entre NOT y OR) y XOR (fusión entre AND y OR), se necesita que los datos de entrada sean iguales para que nazca el valor 1.

III. Operadores relacionales

Los operadores relacionales son aquellos que conectan o comparan datos, mediante estos se crea los valores de verdad, es decir que ha estas comparaciones se les da el valor de True o False.

Igual a ($=$): Devuelve verdadero si ambos operandos tienen el mismo valor [3]

Diferente de ($\neq$): Devuelve verdadero si los operandos tienen valores diferentes [3].

Mayor que ($>$): Verdadero si el operando izquierdo es superior al derecho [3]

Menor que ($<$): Verdadero si el operando izquierdo es inferior al derecho.

Mayor o igual que ($\geq$): Verdadero si el izquierdo es mayor o igual al derecho.

Menor o igual que ($\leq$): Verdadero si el izquierdo es menor o igual al derecho.

IV. Estructuras de control de flujo

Las estructuras de control de flujo son aquellas que ponen en marcha a la lógica booleana en el código, entre ellos tenemos:

Estructura selección (if-else):

Evalúa una condición y selecciona entre 2 posibles caminos de ejecución en lenguaje de programación C.

```
#include <stdio.h>
int main () {
    // Variables
    int edad;
    char genero;
    // Datos de entrada
    printf ("Ingrese su edad\n");
    scanf ("%i", &edad);

    printf ("Ingrese su genero M (Mujer) o H (Hombre)\n");
    scanf ("%c", &genero);
    // Proceso
    if (edad >= 18 && (genero == 'M' || genero == 'm')) {
        printf ("\n Resultado Cumple con los requisitos de entrada Bienvenido");
    }
```

```

} else {
    printf ("In Resultado: No cumple con los requisitos para el ingreso");
}
return 0;
}

```

En este programa se puede observar operadores lógicos "AND" y "OR"

OR: Este operador en el programa permite múltiples opciones válidas (genero = 'M' || genero == 'm'). Si representamos estos en la tabla de verdad podemos analizar las siguientes:

M	m	M m	Procesamiento de la computadora
V	V	V	Como la proposición verdadera la computadora la ejecuta o analiza.
V	F	V	Como la proposición verdadera la computadora la ejecuta o analiza como 1 (solo se necesita que una de las opciones se cumpla).
F	V	V	Como la proposición verdadera a la computadora la analiza como 1 (solo si el usuario no ingresa ninguna de las variables).
F	F	F	La proposición es falsa por lo tanto la computadora la analiza como 0 (solo si el usuario no ingresa ninguna de las variables la computadora la analiza como no cumplida).

AND: Presenta

Resumen: En el código "if" significa en español "Si", lo que hace esto es que si las proposiciones que están dentro del paréntesis es verdadero (lo cual está definido por los conectores lógicos, en este caso AND y OR) se lleva a cabo una acción determinada pero si la primera condición no se cumple pasa a la acción determinada por "else" imprimiendo dicha opción.

Estructura de repetición (while):

Ejecuta una condición mientras sea verdadera y solo se detiene cuando esta sea falsa.

En el lenguaje de programación C se representa de la siguiente manera:

```

#include <stdio.h>
int main () {
    int clave = 0;
    while (! (clave == 7)) {
        printf ("Escribe la clave secreta para salir: ");
        scanf ("%i", &clave);
    }
    printf ("¡Clave correcta! Saliendo...");
    return 0;
}

```

ESTILO

En este programa se puede ver conector lógico NOT

NOT: Este operador lógico permite cambiar el valor de la verdad a una proposición, en el programa permite que While (Ciclo) siga funcionando ya que dice que si la clave es diferente a 7 tendrá el valor de verdad de Falso por lo que el ciclo terminará, pero si usamos ! la proposición sería verdadera por lo que el ciclo continúa hasta que la clave sea 7 (verdadera) y por el no se convierta en F concluyendo con el ciclo. Tabla de verdad:

Clave	Clave == 7	!(Clave == 7)
7	V	F (0)
Cualquier otro número	F	V (1)

Análisis de la Lógica Booleana
La computadora detecta que la condición de parada se cumple, el invierte el éxito a "falso" y el ciclo se detiene.
La computadora ve que la igualdad no se cumple, el ! lo cumple "verdadero" y el ciclo continúa repitiéndose.

1. Explique la diferencia entre una proposición simple y una proposición compuesta.
La proposición simple es una sola oración sin conectores lógicos a diferencia de las proposiciones compuestas que está conformada por 2 o más oraciones (proposiciones simples) que están conectadas por conectores lógicos (no, y, o, si... entonces, si... solo si).
Ambas son importantes en la programación ya que son base del pensamiento computacional y la toma de decisiones en los algoritmos, pero al no identificarlos correctamente puede ocasionar errores de ejecución, deficiencia en los programas y dificultades en la lectura y procesamiento, esto se debe a que las proposiciones simples definen condiciones básicas y la compuestas contruye logica compleja. Proposición simple
Es una proposición simple ya que es una comparación

```
#include <stdio.h>
int main () {
    // Variables
    int edad;
    // Datos de entrada
    printf ("ingrese su edad \n");
    scanf ("%i", &edad);
    // Una proposición simple es una sola comparación
    if (edad < 18)
        printf ("Usted es menor de edad \n");
    return 0;
}
```

Este programa es una proposición compuesta porque has dos comparaciones a variables ("edad >= 18" y "tutor[edad] == 's'") y estas a su vez se relacionan mediante "o" (||) que es un conector lógico.

2. Realiza grupo: cada grupo recibe ejercicios de simplificación de expresiones lógicas aplicando las leyes fundamentales (ley de identidad, complemento, idempotencia, asociatividad, distributiva de Morgan, absorción).

PRIMER EJERCICIO

$$[(p \vee q) \wedge q] \vee \neg p$$

$[(p \wedge q) \vee (\neg q \wedge q)] \vee \neg p$ Ley distributiva
 $[(p \wedge q) \vee \neg p] \vee \neg p$ Ley de complemento
 $(p \wedge q) \vee \neg p$ Ley de identidad
 $(\neg p \vee q) \wedge (\neg p \vee q)$ Ley distributiva
 $\neg p \wedge (\neg p \vee q)$ Ley de complemento
 $\neg p \vee q$ Ley de identidad

Tabla de verdad:

$[(p \vee \neg q) \wedge q] \vee \neg p$					
$\neg$	$\vee$	$\neg$	$\wedge$	q	$\vee \neg p$
$\vee$	$\vee$	$\neg$	$\vee$	$\vee$	$\vee \neg$
$\vee$	$\vee$	$\vee$	$\neg$	$\neg$	$\neg \neg$
$\neg$	$\neg$	$\neg$	$\neg$	$\vee$	$\vee \vee$
$\neg$	$\vee$	$\vee$	$\neg$	$\neg$	$\vee \vee$

SEGUNDO EJERCICIO

$$[(p \vee q \vee r) \vee (\neg r \wedge q \wedge p)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$$

$[(\neg r \wedge q \wedge p) \vee (r \wedge q \wedge p)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$ Ley de Morgan
 $\neg \neg [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$ Ley de complemento
 $\neg \neg [(q \wedge r) \vee (q \wedge \neg r)] \vee (\neg q \wedge r) \leftrightarrow (q \vee r)$ Ley de asociatividad
 $\neg \neg [(q \wedge (r \vee \neg r)) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$ Ley distributiva
 $\neg \neg [(q \wedge 1) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$ Ley de complemento
 $\neg \neg [(q \vee (\neg q \wedge r))] \leftrightarrow (q \vee r)$ Ley de absorción
 $\neg \neg [(q \vee \neg q) \wedge (q \vee r)] \leftrightarrow (q \vee r)$ Ley distributiva
 $\neg \neg [1 \wedge (q \vee r)] \leftrightarrow (q \vee r)$ Ley de complemento
 $\neg \neg [q \vee r] \leftrightarrow (q \vee r)$
 $(q \vee r) \leftrightarrow (q \vee r)$
 $\vee$
 1

Es una contingencia

Quinto Ejercicio

$$(p \vee q) \wedge (p \vee q) \rightarrow q$$

$$q \vee (p \wedge p) \rightarrow q \text{ ley distributiva}$$

$$q \vee p \rightarrow q \text{ ley de complemento}$$

$$q \rightarrow q \text{ ley de identidad}$$

$$\neg q \vee q \text{ ley de complemento.}$$

V

tabla de verdad.

$$p \quad q \quad \neg p \vee q \quad (p \vee q) \quad (p \vee q) \wedge (p \vee q) \quad (p \vee q) \wedge (p \vee q) \rightarrow q$$

V	V	F	V	V	V
V	F	V	V	V	V
F	V	V	V	V	V
F	F	V	V	V	V

V ≠ V ≠

V > > >

Es una tautología.

3. Los computadores utilizan patrones de lógica computacional como las expresiones booleanas, se utilizan en la programación de microcontroladores, en programación de PLC, y en otros dispositivos lógicos en electrónica.

Concepto de Lógica Booleana

George Boole (1815-1864) en lógica booleana, descrita por George Boole en el siglo XIX, es una forma de algebra donde una variable proposicional como variable algebraica, solo puede tener dos valores: Verdadera (V) o Falsa (F).

Se relacionamos este concepto con programación porque desde que en la lógica booleana el resultado de cualquier computadora no podemos analizar los algoritmos, ya que es el puente entre lenguaje humano y el de la computadora (Esta solo entiende 1-True y 0-False).

Operadores lógicos

Como permiten evaluar condiciones actuando como conectivos entre expresiones.

= AND (y/∧): Funciona como una multiplicación matemática. El resultado será 1 (Verdadero) si cualquiera de las sentencias de entrada es 1.

= NOT (no/¬): Invierte el valor lógico. Si entre un 1, sale 0, si entra un 0, sale un 1.

NOTA: Existen operadores combinados como el NAND (NOT + AND), el NOR (NOT + OR) y el XOR (el resultado es 1 si los operandos son diferentes entre sí).

Operadores Relacionales

Son aquellos que se utilizan para comparar o comparar datos, generando valores de verdad (Verdadero o Falso) según el resultado de la comparación.

Relación	Simbolización
Igual a	=
Distinto de	≠
Mayor que	>
Menor que	<
Mayor o Igual que	>=
Menor o Igual que	<=



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación



Resumen: En el código `if` significa un espacio `Si`, lo que hace esto es que si las condiciones que están dentro del paréntesis no son verdaderas (lo cual está definido por los operadores lógicos, en caso `AND` y `OR`) se lleva a cabo una acción determinada pero si la primera condición no se cumple pasa a la acción determinada por `else` imprimiendo dicha opción.

Estructura de repetición (while)

Ejecuta una condición mientras sea verdadera y solo se detiene cuando esta sea falsa.

Lenguaje de programación C.

```
#include <stdio.h>
int main() {
    int clave = 0;

    while (! (clave == 7)) {
        printf("Escriba la clave secreta para salir = ");
        scanf("%i", &clave);
    }

    printf("! Clave correcta ¡ Saliendo... ");

    return 0;
}
```

LUT - Esta operación lógica permite cambiar el valor de la verdad a una proposición, en el programa permite que while (false) siga funcionando ya que como el la clave es diferente a 7 invierte el valor por lo que el ciclo continúa de falso por lo que el ciclo permanece, pero en un momento la proposición se vuelve verdadera por lo que el ciclo continúa hasta que la clave sea 7 (verdadero) y por el no se encuentra en el condicional con el ciclo Tabla de verdad.

Clave	$clave == 7$	$!(clave == 7)$	Análisis de la lógica Booleana
7	V	F(0)	La computadora detecta que la condición de parada se cumplió, el $!$ invierte el valor a "falso" y el ciclo se detiene.
Cualquier otro número	F	V(1)	La computadora ve que la igualdad no se cumple, el $!$ la cambia a "verdadero" y el ciclo continúa indefinidamente.



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Yuleisy Cecibel Jaramillo Jaramillo

UNIVERSIDAD NACIONAL DE LOJA
FEIRNNR
Carrera de Computación
APE 2

Nombre: Yuleisy Cecibel Jaramillo Jaramillo
Asignatura: Matemáticas Discretas
Ciclo: 1er A
Nombre del docente: Mario Enrique Cueva Hurtado
Fecha: 13 de Abril al 17 de Abril de 2026

1.- Los estudiantes construyen tablas de verdad para expresiones con 2, 3 y 4 variables proposicionales, identificando los casos en que la expresión resulta verdadera, falsa o es una tautología / contradicción / contingencia.

Ejercicios de 2 variables:

a) $(p \wedge \neg q) \vee (\neg p \wedge (p \vee q))$

R: El ejercicio es una contingencia.

$(p \wedge \neg q)$	$\vee$	$(\neg p \wedge (p \vee q))$
V F F V	F	F V F V V V
V V V F	V	F V F V V F
F F F V	V	V F V F V V
F F V F	F	V F F F F F

b) $(p \vee q) \wedge (\neg p) \wedge (\neg q)$

R: El ejercicio es una contradicción.

$(p \vee q)$	$\wedge$	$(\neg p)$	$\wedge$	$(\neg q)$
V V V	F	F V	F	F V
V V F	F	F V	F	V F
F V V	F	V F	F	F V
F F F	F	V F	V	V F

Ejercicios de 3 variables:

c) $(p \vee q) \wedge (\neg p \vee q) \wedge (\neg q \vee r) \wedge \neg r$

$(p \vee q)$	$\wedge$	$(\neg p \vee q)$	$\wedge$	$(\neg q \vee r)$	$\wedge$	$\neg r$
VVV	V	FVVV	V	FVVV	F	FV
VVV	V	FVVV	F	FVFF	F	FV
VVF	F	FVFF	F	VFVV	F	FV
VVF	F	FVFF	F	VFVF	F	FV
FVV	V	VFVV	V	FVVV	F	FV
FVV	V	VFVV	F	FVFF	F	FV
FFF	F	VFVF	F	VFVV	F	FV
FFF	F	VFVF	F	VFVF	F	FV

R: Este ejercicio es una contradicción

d) $[(p \rightarrow q) \wedge (q \rightarrow r)] \rightarrow (p \rightarrow r)$

$[(p \rightarrow q) \wedge (q \rightarrow r)]$	$\rightarrow$	$(p \rightarrow r)$
VVV	V	VVV
VVF	V	VVF
VFF	F	VVV
VFF	F	VFF
FVV	V	FVV
FVF	V	FVF
FVF	V	FVV
FVF	V	FVF

R: Este ejercicio es una tautología

2. Practica grupal: cada equipo resuelve ejercicios de simplificación de expresiones lógicas aplicando las leyes fundamentales (ley de identidad, complemento, idempotencia, conmutativa, asociativa, distributiva, de Morgan, absorción).

Primer ejercicio.

$$\begin{array}{l} [p \vee \neg q] \wedge q \quad \neg \neg p \\ [p \wedge (q \wedge \neg q)] \vee \neg p \\ [p \wedge \neg q] \vee \neg p \\ (p \wedge \neg q) \vee \neg p \\ (p \vee p) \wedge (p \vee \neg q) \\ \quad \quad \quad \quad \quad \quad \neg p \vee q \\ \quad \quad \quad \quad \quad \quad \neg p \vee q \end{array}$$

ley distributiva
ley de complemento
ley de identidad
ley distributiva
ley de complemento
ley de identidad

R: Es una contingencia

E	(	p	v	1	q	)	^	q	]	v	1	p
		v	v	f	v		v	v		v	f	v
		v	v	v	f		f	f		f	f	v
		f	f	f	v		f	v		v	v	f
		f	v	v	f		f	f		v	v	f

7	8	9
F	V	V
F	V	F
V	F	V
V	F	F

Segundo ejercicio

[illegible]

1. Ley de Morgan
2. Ley de Complemento
3. Ley asociativa
4. Ley distributiva

- 5 Ley de complemento
- 6 Ley de propiedad
- 7 Ley distributiva
- 8 Ley de complemento

Es una tautología

[illegible]

Tensor ejaculo,

$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	1
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	1
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	2
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	2
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	3
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	1
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	2
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	1
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	1
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	4
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	2
$\neg((p \wedge q) \vee (p \wedge r)) \vee (r \wedge r) \vee ((p \vee q) \wedge (p \vee r))$	

Es una tautología

1. Ley distributiva
2. Ley de complemento
3. Ley asociativa
4. Ley de absorción

3.- Los equipos analizan problemas de lógica computacional como las expresiones booleanas se aplican en la evaluación de condiciones en programación (if-else, while operadores lógicos en Python / C / Java).

Concepto de lógica booleana

Según Días [13] "En lógica booleana, desarrollada por George Boole en el siglo XIX, es una forma de álgebra donde una variable proposicional como variable discreta, solo puede tener dos valores: Verdadera (V) o Falsa (F).

Si relacionamos este concepto con programación podemos decir que sin la lógica booleana el software de cualquier computadora no podría analizar los algoritmos, ya que, es el puente entre lenguaje humano y el de la computadora (la cual solo entiende 1 que es True y 0 que es False).

Operadores lógicos

Los operadores lógicos permiten evaluar diferentes condiciones actuando como conectores entre dos expresiones ya unidas por operadores racionales.

AND (&&): Dos o más señales de entrada y aplicarle la función matemática de multiplicación. Para obtener un 1 lógico de salida, todas las señales de entrada deben ser 1 lógico.

OR (|| / |): Toma dos o más señales de entrada y aplicarle la función matemática de suma, para obtener un 1 lógico de salida, cualquiera las señales de entrada deben de ser un 1 lógico.

NOT (! / NO): Su funcionalidad se basa en cambiar su valor lógico, decir si la señal de entrada es un 1 lógico, al aplicarle la función NOT se obtendría un cero lógico y así a la inversa.

Nota: Estos son los principales operadores lógicos sin embargo, estos se pueden unir para formar otros operadores lógicos como: NAND (fusión entre NOT y AND) el NOR (fusión entre NOT y OR) y XOR (fusión entre AND y OR, se necesita que los datos de entrada sean iguales para que nos de el valor 1).

Operadores relacionales

Los operadores relacionales son aquellos que conectan o comparan datos mediante estos se crean los valores de verdad.

- Igual a ($=$)
- Distinto de ($\neq$ o $<>$)
- Mayor que ($>$)
- Menor que ($<$)
- Mayor o igual que ($\geq$)
- Menor o igual que ($\leq$)

Estructuras de control de flujo
Las estructuras de control de flujo son aquellos que ponen en marcha a la lógica booleana en el código, entre ellos tenemos:

Estructura selección (if-else):
Evalúa una condición y selecciona entre dos posibles caminos de ejecución.

Lenguaje de programación C

```
#include <stdio.h>
int main() {
    // Variables
    int edad;
    char genero;

    // Datos de entrada
    printf("Ingrese su edad \n");
    scanf("%i", &edad);

    printf("Ingrese su genero M (Mujer) o H (Hombre) \n");
    scanf("%c", &genero);

    // Proceso
    if (edad >= 18 && (genero == 'M' || genero == 'm')) {
        printf("\n RESULTADO: Cumple con los requisitos de entrada - BIENVENIDA \n");
    } else {
        printf("\n RESULTADO: No cumple con los requisitos para el ingreso de esta reunion");
    }

    return 0;
}
```

En este programa se puede observar dos operadores lógicos "AND" y "OR".

OR: Este operador en el programa permite múltiples opciones válidas (genero "M" || genero == "m"). Si representáramos esto en la tabla de verdad podríamos analizar las siguientes posibilidades:

M	m	M m	Procesamiento de la computadora
V	V	V	Como la proposición es verdadera la computadora lo ejecuta o analiza como 1
V	F	V	Como la proposición es verdadera la computadora lo ejecuta o analiza como 1 (solo se necesita que una de las opciones se cumpla).
F	V	V	
F	F	F	La proposición es falsa por lo que la computadora lo analiza como 0 (solo si el usuario no ingresa ninguna de las variables la computadora lo analiza como no cumplida).

AND: Necesita que las dos variables sean verdaderas para que toda la proposición sea verdadera.

(edad >= 18 && (genero == 'M' || genero == 'm')).

edad	genero	edad && genero	Interpretación
V	V	V	Solo cuando las variables cumplen con las condiciones establecidas la proposición se vuelve verdadera y la computadora lo analiza como 1.
V	F	F	Una de las variables no cumple con las condiciones establecidas por lo que la es falsa por lo que la computadora lo analiza como 0.
F	V	F	
F	F	F	Una de las variables no cumple con las condiciones establecidas por lo que la proposición es falsa por lo tanto la computadora lo analiza como 0.

Resumen: En el código "if" significa en español "Si", lo que hace esto es que si las proposiciones dentro del paréntesis es verdadero se lleva a cabo una acción determinada pero si la primera condición no se cumple pasa a la acción determinada por "else" imprimiendo dicha acción.

Estructura de repetición (while):

Ejecuta una condición mientras sea verdadera y solo se detiene cuando esta sea falsa.

Lenguaje de programación C:

```
#include <stdio.h>
int main() {
    int clave = 0;

    while (! (clave == 7)) {
        printf("Escribe la clave secreta para salir: ");
        scanf("%d", &clave);
    }

    printf("! Clave correcta! Saliendo...");
    return 0;
}
```

NOT: Este operador lógico permite cambiar el valor de la verdad a una proposición, en el programa permite que while (ciclo) siga funcionando ya que si dice que si la clave es diferente a 7 tendrá el valor de verdad FALSO por lo que el ciclo terminaría, pero si usamos ! la proposición sería verdadera y el ciclo continuaría hasta que la clave sea 7 (Verdadera) y, por lo que se convierte F concluyendo el ciclo.

clave	clave == 7	!(clave == 7)	Análisis de la Lógica Booleana
7	V	F(0)	La computadora detecta que la condición se cumple, el ! invierte el valor a Falso y el ciclo se detiene.
Cualquier otro valor	F	V(1)	La computadora ve que la igualdad no se cumple el ! cambia el valor a Verdadero y el ciclo continúa.



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Sdier Emanuel Roblez Roblez

Universidad Nacional de Loja

Nombre: Sdier Emanuel Roblez Roblez

Ciclo: 1A Fecha:

Docente: Mario Enrique Cueva Hurtado.

Los estudiantes construyen tablas de verdad para expresiones con 2, 3 y 4 variables proposicionales identificando los casos en que la expresión resulta verdadera, falsa o es una tautología/contradicción/contingencia.

Ejercicios de 2 variables

a) $(p \wedge q) \wedge (\neg p) \wedge (\neg q)$

El ejercicio es una contingencia

$(p \vee q) \wedge (\neg p) \wedge (\neg q)$

V	V	V	F	F	V	F	F	V
V	V	F	F	F	V	F	F	V
F	V	V	F	V	F	F	F	V
F	F	F	F	V	F	V	V	F

b) $(p \wedge \neg q) \vee (\neg p \wedge (p \vee q))$

El ejercicio es una contingencia

$(p \wedge \neg q) \vee (\neg p \wedge (p \vee q))$

V	F	F	V	F	F	V	F	V
V	V	V	F	V	F	V	V	F
F	F	F	V	V	F	V	F	V
F	F	V	F	V	F	F	F	F

Ejercicios de 3 variables

c) $(p \vee q) \wedge (\neg p \vee q) \wedge (\neg q \vee r) \wedge \neg r$

$(p \vee q) \wedge (\neg p \vee q) \wedge (\neg q \vee r) \wedge \neg r$

V	V	V	V	F	V	V	V	F	V	V	V	F	F	V
V	V	V	V	F	V	V	V	F	F	V	F	F	F	F
V	V	F	F	F	V	F	F	F	V	F	V	V	F	F
V	V	F	F	F	V	F	F	F	F	V	F	V	F	F
F	V	V	V	V	F	V	V	V	V	F	V	V	V	F
F	V	V	V	V	F	V	V	V	F	F	V	F	F	F
F	F	F	F	F	V	F	V	F	F	V	F	V	V	F
F	F	F	F	F	V	F	V	F	F	V	F	F	F	F

Este ejercicio es una contradicción.

d) $[(p \rightarrow q) \wedge (q \rightarrow r)] \rightarrow (p \rightarrow r)$

$[(p \rightarrow q) \wedge (q \rightarrow r)] \rightarrow (p \rightarrow r)$

V	V	V	V	V	V	V	V	V	V	V	V	V	V	V
V	V	V	F	V	F	F	F	V	V	F	F	F	F	F
V	F	F	F	F	V	V	V	V	V	V	V	V	V	V
V	F	F	F	F	V	F	F	F	V	F	F	F	F	F
F	V	V	V	V	V	V	V	V	V	F	V	V	V	V
F	V	V	F	V	F	F	F	V	V	F	V	F	F	F
F	V	F	V	V	F	V	V	V	V	F	V	V	V	V
F	V	F	V	V	F	V	F	F	V	F	V	F	F	F

Este ejercicio es una tautología.

$$[(p \vee \neg q) \wedge q] \vee \neg p$$

U	V	F	V	V	U	V	F	V
V	V	U	F	F	F	F	F	V
F	F	F	V	F	V	V	U	F
F	V	V	F	F	F	V	V	F

$$[(p \vee q \vee r) \vee (\neg p \wedge q \wedge \neg r)] \wedge [(q \vee r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \rightarrow (q \vee r)$$

$$[(p \vee q) \vee r] \wedge [(q \wedge r) \vee (r \wedge p)] \quad \text{Morgan}$$

$$\wedge \vee [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \quad \text{Complemento}$$

$$(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)$$

identidad

$$[q \wedge (r \vee \neg r)] \vee (\neg q \wedge r)$$

distributiva

$$(q \wedge v) \vee (\neg q \wedge r)$$

complemento

$$q\psi(-q, \lambda, \gamma)$$

identical

$$(qv \vdash q) \wedge (qv \vdash r)$$

distributiva

$$\forall x \wedge (p \vee r)$$

complements

Q v Y

identidad

$$[(p \vee q \wedge r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$$

VVVVVV VFFFFF V VVVVV VEEV FEV V VVV
VVVVV VFFFFF V VEEV VVVVV VFF V VVV
VVVVV VEEVVV V FEVFFFFF VVV V FVV
VVVVV VEEVVV F FFFFFFVF VFF V FFF
FVVVV VFFFFF V VVVVVVFVV FFEV V VVV
FVVVV VVVVV V VEEVVVVVV FFF V VVV
FFFEV VVVVV V FEEVVFFF VVV V FVV
FFEFF VVVVV F FFFFFFVF VFF V FFF

Tender specimen:

$$[(p \wedge q) \vee (p \wedge \neg q)] \vee (r \wedge \neg r) \vee [(p \vee q) \wedge (p \vee \neg q) \wedge (p \vee r)]$$

$$[(p \wedge (q \vee r)) \vee (q \wedge r)] \vee [(p \vee q) \wedge (p \vee r) \wedge (p \vee q) \wedge (p \vee r)]$$

$$[r(p \wedge 1 \vee 0)] \vee [(p \vee q) \wedge (p \vee q) \wedge (p \vee r)] \quad 2$$

$$[p \vee q] \vee [(p \wedge q) \wedge (p \vee q) \wedge (p \vee r)]$$

$$\neg p \vee [(p \vee q) \wedge (p \vee \neg q) \wedge (p \vee r)]$$

$$p \vee [p \vee (q \wedge r) \wedge (p \vee r)]$$

$$\tau_p \vee [(\text{pro}) \wedge (\text{pur})] \quad 2$$

$p \vee [p \wedge (p \vee r)]$ 1

$$\neg p \vee [(p \wedge q) \vee (p \wedge r)]$$

$$\gamma_p \in [\rho_A(\rho_{A^c})]$$

Es una lautología

1. ley distributiva
2. ley de complemento
3. ley de absorción
4. ley asociativa

$$\vdash ((p \wedge q) \vee (p \wedge \neg q)) \vee (\neg p \wedge r) \vdash [(p \vee q) \wedge (p \vee \neg q) \wedge (p \vee r)]$$

[illegible]

Cuarto ejercicio

$$[(p \wedge \neg q) \vee p] \wedge \neg(\neg p \vee q)$$

$$[p \wedge (\neg p \vee q)] \wedge \neg(\neg p \vee q)$$

$$[p \wedge (p \vee \neg p)] \wedge \neg(p \wedge \neg q)$$

$$p \wedge (p \wedge \neg q)$$

$$(p \wedge p) \wedge \neg q$$

$$p \wedge \neg q$$

Ley de absorción

Ley de Morgan

Ley asociativa

Ley de idempotencia

Es una contingencia

$$[(p \wedge \neg q) \vee p] \wedge \neg(\neg p \vee q)$$

V	F	F	V	V	V	F	F	F	V	V	V
V	F	V	F	V	V	V	V	F	V	F	F
F	F	F	V	F	F	F	F	V	F	V	V
F	F	V	F	F	F	F	F	V	F	V	F

Quinto ejercicio:

$$(\neg p \vee q) \wedge (p \vee q) \rightarrow q$$

$$q \vee (\neg p \wedge p)$$

$$q \vee F$$

$$q \rightarrow q$$

$$\neg q \vee q$$

$$V$$

Ley distributiva

Ley de complemento

Ley de identidad

Ley de complemento

$$(\neg p \vee q) \wedge (p \vee q) \rightarrow q$$

F	V	V	V	V	V	V	V
F	V	F	F	F	V	V	F
F	F	V	V	V	F	V	V
F	F	V	F	F	F	V	F



Universidad nacional de Loja

Nombre: Sdier Emanuel Roblez Roblez Fecha: 28/04/2026

Docente: Mario Enrique Cueva Hurtado

3. Los equipos analizan problemas de lógica computacional: cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación.

Concepto de lógica booleana.

Según Díaz [1] "En la lógica booleana, desarrollada por George Boole en el siglo XIX, es una forma de álgebra donde una variable proposicional, como variable discreta, solo puede tener dos valores: Verdadera (V) o Falsa (F)". En programación es fundamental porque permite que las computadoras interpreten algoritmos mediante valores binarios (1 = verdadero y 0 = falso).

Los operadores lógicos se utilizan para evaluar condiciones y relacionar expresiones. Los principales son:

- AND (&&): produce un resultado verdadero solo si todas las condiciones son V.
 - OR (||): produce un resultado verdadero si al menos una condición es verdadera.
 - NOT (!): invierte el valor lógico, cambiando verdadero por falso y viceversa.
- Estos operadores también pueden combinarse para formar otros como NAND, NOR, XOR.

Por otro lado, los operadores relacionales se utilizan para comparar datos y generar valores de verdad. Algunos ejemplos son: igual que (==), mayor que (>), menor que (<) y mayor o igual que (>=).

1. Ejemplos de lenguaje de programación C.

Estructura de selección (if-else)

```
#include <stdio.h>
```

```
int main () {
```

```
    // variables
```

```
    int edad;
```

```
    char genero;
```

```
    // Datos de entrada
```

```
    printf ("Ingrese su edad \n");
```

```
    scanf ("%i",&edad);
```

```
    printf ("Ingrese su genero M (Mujer) o H (Hombre) \n");
```

```
    scanf ("%C",&genero);
```

```
    // proceso
```

```
    if (edad >= 18 && (genero == 'M' || genero == 'm')) {
```

```
        printf ("\nRESULTADO: Cumple con los requisitos de entrada.  
BIENVENIDA \n");
```

```
    } else {
```

```
        printf ("\nRESULTADO: No cumple con los requisitos para el ingreso
```

```

de esta reunión");
}
return 0;
}

```

En este programa se puede observar dos operadores lógicos.
"AND" y "OR"

OR: Este operador en el programa permite opciones validas
Si se representa en la Tabla de verdad:

M	m	M m	<u>Procesamiento de la computadora</u>
V	V	V	Como la proposición es verdadera la computadora lo ejecuta o analiza como 1.
V	F	V	Como la proposición verdadera la computadora lo ejecuta o analiza como 1. (solo se necesita que una de las opciones se cumpla)
F	V	V	Como la proposición verdadera la computadora lo ejecuta o analiza como 1. (solo se necesita que una de las opciones se cumpla)
F	F	F	La proposición es falsa por lo que la computadora lo analiza como un 0 (solo si el usuario no ingresa ninguna de las variables la computadora lo analiza como no cumplida).

AND: Necesita que las dos variables sean verdaderas para que toda la proposición sea verdadera.

edad	genero	edad & genero	Interpretación
V	V	V	Solo cuando las dos variables cumplen con las condiciones establecidas la proposición se vuelve verdadera y la computadora la analiza como 1.
V	F	F	Una de las variables no cumple con las condiciones establecidas por lo que la proposición es falsa por lo que la computadora lo analiza como 0.
F	V	F	Una de las variables no cumple con las condiciones establecidas por lo que la proposición es falsa por lo que la computadora lo analiza como 0.
F	F	F	Una de las variables no cumple con las condiciones establecidas por lo que la proposición es falsa por lo que la computadora lo analiza como 0.

En resumen en el código "if" significa en español "si", lo que hace esto es que si las proposiciones que están dentro del paréntesis es verdadero en este caso AND y OR se lleva a cabo una acción determinada pero si la primera condición no se cumple pasa a la acción determinada por "else" imprimiendo dicha acción.

Estructura de repetición (while):

```
#include <stdio.h>
int main () {
    int clave = 0;

    while (! (clave == 7)) {
        printf("Escriba la clave secreta para salir: ");
        scanf("%i", &clave);
    }

    printf("!Clave correcta; Saliendo...");

    return 0;
}
```

En este programa se puede ver el conector lógico NOT

El operador lógico NOT (!) cambia el valor de verdad de una proposición a su contrario. En el programa, se utiliza en el ciclo while para invertir la condición y permitir que el ciclo continúe ejecutándose mientras la clave sea diferente de 7, finalizando cuando la clave sea 7 y la condición se vuelva falsa.



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Clave	Clave == 7	!(clave == 7)	Análisis de la Lógica Booleana
7	V	F(0)	La computadora detecta que la condición de parada se cumplió, el ! invierte el éxito a "Falso" y el ciclo se detiene.
Cualquier otro número	F	V(1)	La computadora ve que la igualdad no se cumple, el ! lo cambia a "verdadero" y el ciclo continúa repitiéndose.